MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Report

on Laboratory Work no. 8
Operating Systems
Theme: The Same Calls, Inside Your Own OS

Done by: st. gr. FAF-242, **Moroșanu Ion**
Checked by: **Reițman Patricia**, *university assistant*
FCIM, UTM

Chișinău – 2026

TABLE OF CONTENTS

INTRODUCTION **3**
Purpose of the work 3
1 RUN INSIDE XV6 ..... **4**
1.1 The program and the build 4
1.2 The child becomes echo 5
1.3 The child becomes ls, and a program that does not exist 5
1.4 Answers to the questions 7
2 PINGPONG **8**
2.1 The program 8
2.2 The result 8
2.3 Optional: primes 9
2.4 Answers to the questions 9
3 THE EARLIER PROGRAMS ON THE CURRENT XV6 ..... **11**
CONCLUSIONS **12**
REFERENCES **13**

INTRODUCTION

Purpose of the work

In the seventh laboratory work the system calls fork, exec and wait were used on Linux. The purpose of this work is to run the same program inside xv6, the small operating system built during the semester, and to see that the calls are the same, and then to make two processes talk to each other through pipes [1].

The following tasks were set for the work:

a) to write user/run.c, the xv6 version of run.c, and to run it with echo, with ls and with a program that does not exist;

b) to write user/pingpong.c, in which a parent and a child send one byte to each other through two pipes;

c) optionally, to write user/primes.c, which prints the prime numbers up to 35 with a chain of processes connected by pipes.

The work was done in ~/xv6-riscv on the same Ubuntu 26.04 virtual machine. Before starting, the tree was updated with `git pull` to the current version of xv6, commit f780d78 of 15 October 2026 [2]; the programs of the earlier laboratory works were kept. The virtual machine had 2 virtual processors in this session, and the default 3 harts of xv6 boot very slowly on them, so xv6 was started with `make qemu CPUS=1`. To keep the terminal fast, the build itself was run separately, with its output sent to /tmp/build.log, and only its exit code was printed. The files run.c, pingpong.c, primes.c and the Makefile with the UPROGS lines are in the folder lab8/ of the repository, together with notes.md.

1 RUN INSIDE XV6

1.1 The program and the build

The file user/run.c (Figure 1.1) follows the same three steps as run.c of the seventh work. There is no C library in xv6, so the program includes kernel/types.h and user/user.h and calls the system calls of the kernel directly. The child becomes echo with the arguments hello from the child. Three lines were added to the list UPROGS of the Makefile with one sed command, for run and for the programs of the second part. `cat -A` shows that each of them starts with a tab (^I) and ends with a backslash right before the end of the line ($).

```
wancea@OS-Lab1-2:~/xv6-riscv$ sed -i 's|^\t\$U/_find\\$|&\n\t$U/_run\\\n\t$U/_pingp
ong\\\n\t$U/_primes\\|' Makefile
wancea@OS-Lab1-2:~/xv6-riscv$ cat user/run.c
#include "kernel/types.h"
#include "user/user.h"

// the three steps of the shell, inside xv6:  echo hello from the child
int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());

  int pid = fork();                    // step 1: copy myself
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }

  if(pid == 0){                        // only the child runs this
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);                // step 2: become echo
    printf("exec failed\n");           // runs only if exec failed
    exit(1);
  }

  int status;
  wait(&status);                       // step 3: wait for the child
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
wancea@OS-Lab1-2:~/xv6-riscv$ grep -n -A3 '_find' Makefile | cat -A
139:^I$U/_find\$
140-^I$U/_run\$
141-^I$U/_pingpong\$
142-^I$U/_primes\$
wancea@OS-Lab1-2:~/xv6-riscv$
```

Figure 1.1 – user/run.c and the new lines of UPROGS

The build ended with the exit code 0, and the list of the compiled programs taken from the build log contains user/_run, user/_pingpong and user/_primes next to the programs of the earlier works (Figure 1.2).

```
wancea@OS-Lab1-2:~/xv6-riscv$ git log --oneline -1
f780d78 (HEAD -> riscv, origin/riscv, origin/HEAD) build with -fno-delete-null-poin
ter-checks
wancea@OS-Lab1-2:~/xv6-riscv$ make clean >/dev/null; make kernel/kernel fs.img > /t
mp/build.log 2>&1; echo exit=$?; printf "\e[42m%80s\e[0m\n" ""
exit=0
```

```
wancea@OS-Lab1-2:~/xv6-riscv$ grep -o "user/_[a-z]*" /tmp/build.log | sort -u | tr
"
> " " "; echo
user/_args user/_cat user/_dorphan user/_echo user/_find user/_forktest user/_forph
an user/_grep user/_grind user/_init user/_kill user/_ln user/_logstress user/_ls u
ser/_mkdir user/_mystat user/_pingpong user/_primes user/_rm user/_run user/_sh use
r/_sleep user/_stressfs user/_sync user/_usertests user/_wc user/_zombie
```

Figure 1.2 – The build of the current xv6 with the new programs

1.2 The child becomes echo

After the boot, run printed the same four lines as in the handout (Figure 1.3). The parent has the PID 3, because init is 1 and the shell is 2, and the child has the PID 4. The line hello from the child was printed by echo, and the parent saw the status 0.

```
wancea@OS-Lab1-2:~/xv6-riscv$ make qemu CPUS=1
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -
nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=
raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
$
```

Figure 1.3 – run inside xv6: the child becomes echo

1.3 The child becomes ls, and a program that does not exist

The array of arguments and the call of exec were changed to ls with sed, and grep showed the two new lines, 18 and 19, before the build (Figure 1.4). This time the child printed the listing of the root folder, where every program of xv6 lives, from . and README down to the device console, inode 30, and the parent again saw the status 0.

```
wancea@OS-Lab1-2:~/xv6-riscv$ sed -i -e 's/{ "echo", "hello", "from", "the", "child
", 0 }/{ "ls", 0 }/' -e 's/exec("echo", args);/exec("ls", args);/' -e 's/become ech
o/become ls/' user/run.c
wancea@OS-Lab1-2:~/xv6-riscv$ grep -n -e 'args\[\]' -e 'exec(' user/run.c
18:    char *args[] = { "ls", 0 };
19:    exec("ls", args);                // step 2: become ls
wancea@OS-Lab1-2:~/xv6-riscv$ make fs.img > /tmp/build.log 2>&1; echo exit=$?; prin
tf "\e[42m%80s\e[0m\n" ""
exit=0
```

```
xv6 kernel is booting

init: starting sh
$ run
parent: my pid is 3
child: my pid is 4
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36760
echo           2 4 35624
sleep          2 5 35312
args           2 6 35304
mystat         2 7 36408
find           2 8 42256
run            2 9 35920
pingpong       2 10 35912
primes         2 11 36576
forktest       2 12 18168
grep           2 13 44216
init           2 14 36008
kill           2 15 35544
ln             2 16 35336
ls             2 17 42912
mkdir          2 18 35600
rm             2 19 35584
sh             2 20 58344
stressfs       2 21 36448
usertests      2 22 206832
grind          2 23 51856
wc             2 24 37760
zombie         2 25 34880
logstress      2 26 37592
forphan        2 27 36384
dorphan        2 28 35832
sync           2 29 34976
console        3 30 0
parent: child 4 ended with status 0
```

Figure 1.4 – The child becomes ls

With the name nosuchprog exec failed, because there is no such file on the disk. The child continued after the call, printed exec failed and ended with exit(1), and the parent reported the status 1 (Figure 1.5). Afterwards user/run.c was restored to the version with echo.

```
wancea@OS-Lab1-2:~/xv6-riscv$ sed -i 's/exec("ls", args);/exec("nosuchprog", args);
/' user/run.c
wancea@OS-Lab1-2:~/xv6-riscv$ grep -n -e 'args\[\]' -e 'exec(' user/run.c
18:    char *args[] = { "ls", 0 };
19:    exec("nosuchprog", args);                // step 2: become ls
wancea@OS-Lab1-2:~/xv6-riscv$ make fs.img > /tmp/build.log 2>&1; echo exit=$?; prin
tf "\e[42m%80s\e[0m\n" ""
exit=0
```

```
wancea@OS-Lab1-2:~/xv6-riscv$ make qemu CPUS=1
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -
nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=
raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ run
parent: my pid is 3
child: my pid is 4
exec failed
parent: child 4 ended with status 1
$
```

Figure 1.5 – exec of a program that does not exist

1.4 Answers to the questions

1. In xv6 run.c uses `fork()`, `exec("echo", args)` and `wait(&status)`. On Linux it used `fork()`, `execlp("ls", "ls", "-l", NULL)` and `waitpid(pid, &status, 0)`. The three steps are the same, and so are their rules: fork returns 0 in the child and the PID of the child in the parent, exec replaces the program of the process and returns only on an error, and wait blocks the parent until the child ends and gives back its exit status. The differences are small. execlp is a function of the C library that searches PATH and takes the arguments as a list, while the xv6 exec takes the name exactly as given and an array of arguments ending with 0. waitpid waits for one chosen child and needs the macro WEXITSTATUS, while the xv6 wait waits for any child and returns the plain number.

2. The line hello from the child is printed by echo. After exec the child process no longer runs the code of run: its memory holds the program echo, and the array args became the argv of echo. The code of run prints only the lines that start with parent: and child:.

3. The status is simply the number that the child passed to exit after the failed exec. The xv6 version of run.c has `exit(1)` on that line, while the Linux version has `exit(127)`, the code that shells use for a command that is not found.

2 PINGPONG

2.1 The program

The program user/pingpong.c (Figure 2.1) creates two pipes: p2c from the parent to the child and c2p from the child to the parent [3]. After fork each process closes the two ends it does not use. The child reads one byte from p2c, prints its PID with received ping, writes the byte back into c2p and exits. The parent writes the byte p into p2c, reads the answer from c2p, prints its PID with received pong and waits for the child. Only the four lines left empty in the handout were filled in, and the byte is set to p before it is sent, because a variable that is never given a value should not be sent.

```
wancea@OS-Lab1-2:~/xv6-riscv$ cp /tmp/run.c.echo user/run.c && grep -n 'exec(' user
/run.c
19:    exec("echo", args);                 // step 2: become echo
wancea@OS-Lab1-2:~/xv6-riscv$ cat user/pingpong.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];     // parent-to-child, child-to-parent
  char b;

  pipe(p2c);
  pipe(c2p);
  if(fork() == 0){
    close(p2c[1]); close(c2p[0]);       // the ends the child does not use
    read(p2c[0], &b, 1);                // read 1 byte from p2c[0] into b
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);               // write the byte b into c2p[1]
    exit(0);
  }
  close(p2c[0]); close(c2p[1]);         // the ends the parent does not use
  b = 'p';
  write(p2c[1], &b, 1);                 // write 1 byte into p2c[1]
  read(c2p[0], &b, 1);                  // read 1 byte from c2p[0] into b
  printf("%d: received pong\n", getpid());
  wait(0);
  exit(0);
}
wancea@OS-Lab1-2:~/xv6-riscv$ make fs.img > /tmp/build.log 2>&1; echo exit=$?; prin
tf "\e[42m%80s\e[0m\n" ""
exit=0
```

Figure 2.1 – The file user/pingpong.c

2.2 The result

The program printed 4: received ping, from the child, and then 3: received pong, from the parent (Figure 2.2). The order is always this one, because the parent cannot print pong before the byte has come back. run, started right after it, now got the PIDs 5 and 6, since xv6 does not reuse a PID.

```
wancea@OS-Lab1-2:~/xv6-riscv$ make qemu CPUS=1
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -
nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=
raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ pingpong
4: received ping
3: received pong
$ run
parent: my pid is 5
child: my pid is 6
hello from the child
parent: child 6 ended with status 0
```

Figure 2.2 – pingpong and run inside xv6

2.3 Optional: primes

The program user/primes.c is a chain of processes. The first process writes the numbers from 2 to 35 into a pipe. Each stage reads the first number p from its pipe, which is always a prime, prints prime p, creates a new pipe and a child, and passes on only the numbers that p does not divide. When its input pipe is empty, a stage closes the next pipe, so the end of the input moves down the chain, and every process waits for its child before it exits. Each process closes the ends it does not need, otherwise the end of file would never arrive and the processes would run out of descriptors. The program printed the eleven primes from 2 to 31 (Figure 2.3).

```
$ primes
prime 2
prime 3
prime 5
prime 7
prime 11
prime 13
prime 17
prime 19
prime 23
prime 29
prime 31
$
```

Figure 2.3 – primes inside xv6

2.4 Answers to the questions

1. A pipe carries data in one direction only, and any process that holds its reading end may read from it. With a single pipe the parent, which writes the ping and then reads, could read back its own byte before the child takes it, and the two processes would not know whose turn it is. Two pipes give each direction its own channel: the parent only writes into p2c and only reads from c2p, and the child does the opposite.

2. The call `read(c2p[0], &b, 1)` in the parent. read on an empty pipe blocks until data arrives, so the parent sleeps in it until the child has written the byte back. In the same way the read of the child waits for the ping. wait(0) at the end only collects the child after it has exited.

3 THE EARLIER PROGRAMS ON THE CURRENT XV6

Because the tree was updated to the current version of xv6, the programs of the earlier laboratory works were checked again in the same session (Figure 3.1). sleep 10 paused and returned, sleep without an argument printed its usage message, args showed that the xv6 shell passes *.c and the quotes unchanged, mystat printed the inode, the links and the size, and find found ./README. On this fresh disk the root folder shows 1 link, because no folder has been created in it yet. In addition, the files of each xv6 folder of the repository (lab2, lab4, lab6 and lab8) were copied into a separate fresh clone of the current xv6 together with the Makefile of that folder, and each of them built without errors.

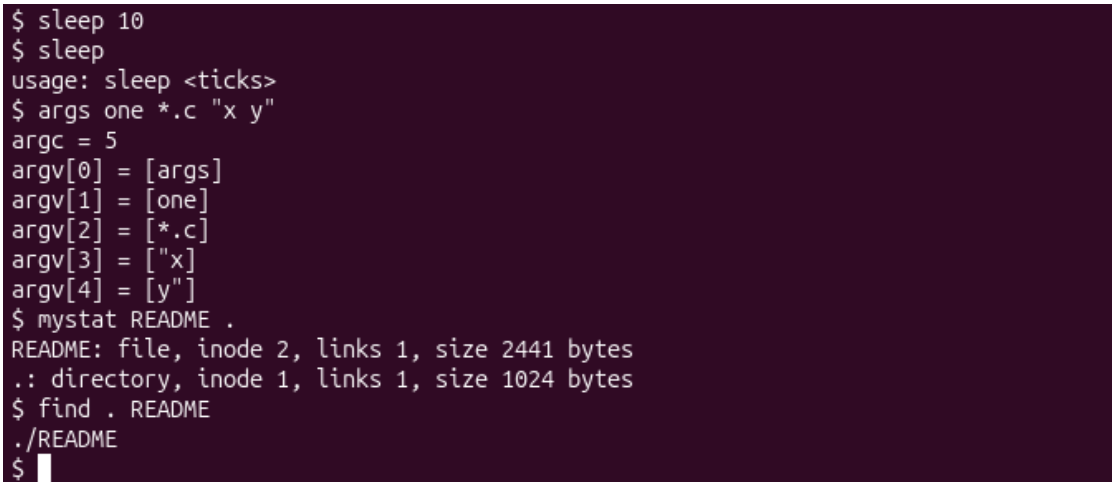


Figure 3.1 – sleep, args, mystat and find on the current xv6

CONCLUSIONS

The work showed that fork, exec and wait are the same three steps in xv6 as on Linux. The program run.c needed only small changes to move from one system to the other, and those changes come from the C library of Linux, not from the kernel: execlp searches PATH and waitpid needs a macro, while the system calls underneath do the same work in both systems. The exit status that the parent sees is nothing more than the number that the child passes to exit, 1 in xv6 and 127 on Linux.

pingpong showed that a pipe is also a way to wait. Nothing in the code tells the parent to wait for the answer, but read on an empty pipe blocks, so the order ping first, pong second is guaranteed by the kernel. Because a pipe goes in one direction, a conversation needs two of them, and each process has to close the ends it does not use. The optional primes program pushed the same idea further: eleven processes connected by pipes, each closing its unused ends so that the end of the input can travel down the chain.

Updating the tree to the current xv6 also confirmed that every program written in the semester, from sleep to primes, still builds and runs on it.

REFERENCES

1. The Same Calls, Inside Your Own OS. Laboratory Work 8, Operating Systems course. Faculty of Computers, Informatics and Microelectronics, Technical University of Moldova, 2026.
2. xv6 for RISC-V. Source code repository, MIT PDOS [online]. Available: https://github.com/mit-pdos/xv6-riscv
3. COX, R., KAASHOEK, F., MORRIS, R. xv6: a simple, Unix-like teaching operating system. MIT, 2024 [online]. Available: https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf